5.3.2 Returning useful errors with Problem Details

In the `MapPost` endpoint of listing 5.3, we checked to see whether an entity with the given `id` already existed. If it did, we returned a `400` response with a description of the error. The problem with this approach is that the client—typically, a mobile app or SPA—must know how to read and parse that response. If each of your APIs has a different format for errors, that arrangement can make for a confusing API. Luckily, a web standard called Problem Details describes a consistent format to use.

DEFINITION Problem Details is a web specification (https://www.rfc-editor.org/rfc/rfc7807.xhtml) for providing machine-readable errors for HTTP APIs. It defines the required and optional fields that should be in the JSON body for errors.

ASP.NET Core includes two helper methods for generating Problem Details responses from minimal APIs: `Results.Problem()` and `Results.ValidationProblem()` (plus their `TypedResults` counterparts). Both of these methods return Problem Details JSON. The only difference is that `Problem()` defaults to a `500` status code, whereas `ValidationProblem()` defaults to a `400` status and requires you to pass in a `Dictionary` of validation errors, as shown in the following listing.

Listing 5.5 Returning Problem Details using `Results.Problem`

```
using System.Collections.Concurrent;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

var _fruit = new ConcurrentDictionary<string, Fruit>();

app.MapGet("/fruit", () => _fruit);
```